

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN II
MODUL 5
“REKURSIF”



DISUSUN OLEH:
BENING PUTRI NARESWARI SUKARNO

109082500211

S1 IF-13-02

DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng

Asisten Praktikum
Muhammad Agha Zulfadhli
Renisa Assyifa Putri

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026

DASAR TEORI

Logika Rekursif adalah suatu fungsi berparameter yang memanggil dirinya sendiri dengan harga parameter yang berbeda. Logika ini dipakai sebagai pengganti proses iterasi. Pada dasarnya rekursif sering digunakan dalam perhitungan matematika, sebagai contoh pertimbangan fungsi factorial dan juga bilangan Fibonacci. Rekursif juga memiliki hubungan erat dengan konsep matematika, terutama dalam definisi fungsi yang bergantung pada nilai sebelumnya. Proses rekursif akan terus berjalan hingga mencapai base case, kemudian hasilnya akan dikembalikan secara bertahap ke pemanggil sebelumnya. Meskipun rekursif membuat kode lebih ringkas dan mudah dipahami, penggunaannya perlu diperhatikan karena dapat memakan memori lebih besar akibat penggunaan stack pemanggilan fungsi.

Selain itu, berdasarkan jurnal dari National Center for Biotechnology Information (NCBI), rekursif dalam ilmu komputer didefinisikan sebagai fungsi yang didefinisikan berdasarkan dirinya sendiri atau memanggil dirinya sendiri dalam proses komputasi. Konsep ini juga berkaitan dengan pembentukan struktur yang berulang, di mana solusi suatu masalah dibangun dari solusi yang serupa namun lebih sederhana. Dalam praktiknya, rekursif digunakan untuk menghasilkan solusi secara bertahap hingga mencapai kondisi tertentu yang menghentikan proses. Oleh karena itu, pemahaman terhadap konsep rekursif sangat penting dalam pengembangan algoritma karena mampu merepresentasikan proses yang berulang secara efisien dan terstruktur.

SOAL GUIDED

1. Guided 1

Source Code:

```
package main

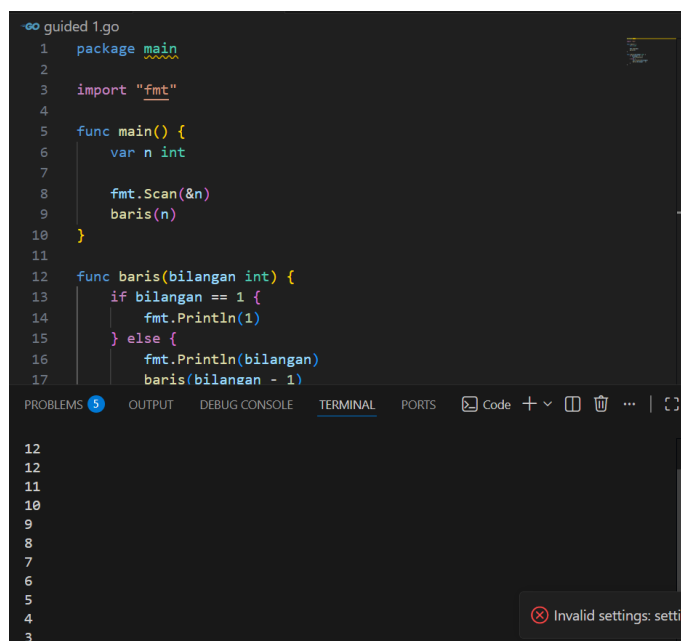
import "fmt"

func main() {
    var n int

    fmt.Scan(&n)
    baris(n)
}

func baris(bilangan int) {
    if bilangan == 1 {
        fmt.Println(1)
    } else {
        fmt.Println(bilangan)
        baris(bilangan - 1)
    }
}
```

Output:



```
guided 1.go
1  package main
2
3  import "fmt"
4
5  func main() {
6      var n int
7
8      fmt.Scan(&n)
9      baris(n)
10 }
11
12 func baris(bilangan int) {
13     if bilangan == 1 {
14         fmt.Println(1)
15     } else {
16         fmt.Println(bilangan)
17         baris(bilangan - 1)
18     }
19 }

12
12
11
10
9
8
7
6
5
4
3

Invalid settings: settin
```

Deskripsi Program:

- Program meminta pengguna menginputkan n .
- Program melakukan menampilkan dari n ke 1, pada gambar pengguna memasukkan angka 12, kemudian program akan melakukan cetak angka dari 12 ke 1.

2. Guided 2

Source Code:

```
package main

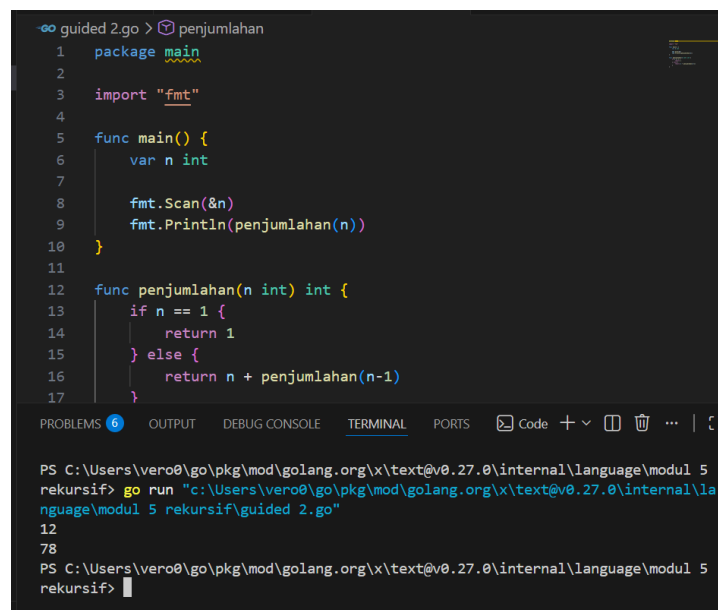
import "fmt"

func main() {
    var n int

    fmt.Scan(&n)
    fmt.Println(penjumlahan(n))
}

func penjumlahan(n int) int {
    if n == 1 {
        return 1
    } else {
        return n + penjumlahan(n-1)
    }
}
```

Output:



The screenshot shows a Go IDE with a dark theme. The editor displays the source code for a recursive function to calculate the sum of numbers from 1 to n. The code is as follows:

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     var n int
7
8     fmt.Scan(&n)
9     fmt.Println(penjumlahan(n))
10 }
11
12 func penjumlahan(n int) int {
13     if n == 1 {
14         return 1
15     } else {
16         return n + penjumlahan(n-1)
17     }
18 }
```

Below the editor, the terminal window shows the command to run the program and its output:

```
PS C:\Users\vero0\go\pkg\mod\golang.org\x\text@v0.27.0\internal\language\modul 5
rekursif> go run "c:\Users\vero0\go\pkg\mod\golang.org\x\text@v0.27.0\internal\la
nguage\modul 5 rekursif\guided 2.go"
12
78
PS C:\Users\vero0\go\pkg\mod\golang.org\x\text@v0.27.0\internal\language\modul 5
rekursif>
```

Deskripsi Program:

- Program menghitung hasil penjumlahan 1 hingga n
- Program menerima input sebuah bilangan n , lalu memanggil fungsi penjumlahan(n).
- Fungsi akan menjumlahkan angka n dengan hasil dari penjumlahan angka sebelumnya ($n-1$) secara terus-menerus hingga mencapai kondisi berhenti, yaitu saat n bernilai 1. Pada kondisi tersebut, fungsi mengembalikan nilai 1.

3. Guided 3

Source Code:

```
package main

import "fmt"

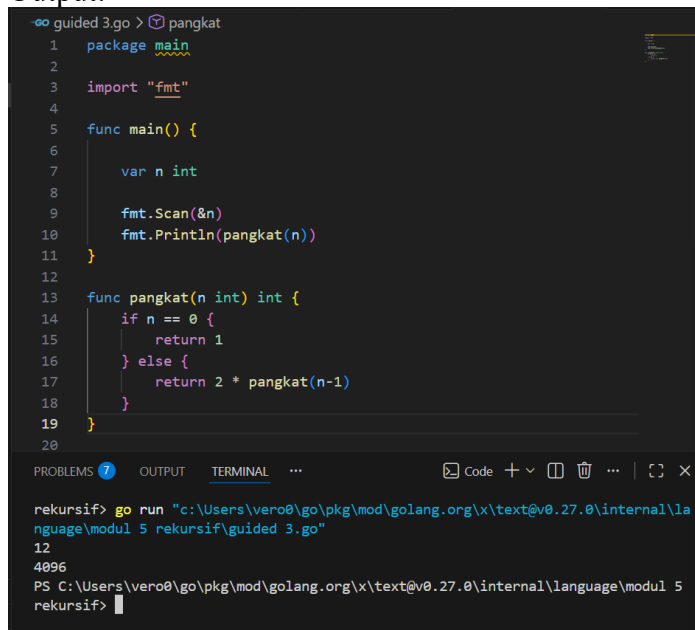
func main() {

    var n int

    fmt.Scan(&n)
    fmt.Println(pangkat(n))
}

func pangkat(n int) int {
    if n == 0 {
        return 1
    } else {
        return 2 * pangkat(n-1)
    }
}
```

Output:



The screenshot shows a Go IDE with the source code from the previous block. The terminal output shows the command `go run "c:\Users\vero0\go\pkg\mod\golang.org\x\text@v0.27.0\internal\language\modul 5 rekursif\guided 3.go"` being executed, followed by the output `12` and `4096`. The prompt `PS C:\Users\vero0\go\pkg\mod\golang.org\x\text@v0.27.0\internal\language\modul 5 rekursif>` is visible at the bottom.

Deskripsi Program:

- Program menghitung nilai 2 pangkat n (2^n) dengan rekursif.
- Program menerima input sebuah bilangan n, lalu memanggil fungsi pangkat(n).
- Fungsi akan mengalikan angka 2 dengan hasil pemanggilan fungsi sebelumnya (pangkat(n-1)) secara berulang hingga mencapai kondisi berhenti, yaitu saat n bernilai 0. Pada kondisi tersebut, fungsi mengembalikan nilai 1.
- Hasil akhirnya adalah nilai 2 dipangkatkan n, kemudian ditampilkan ke layar.

4. Guided 4

Source Code:

```
package main

import "fmt"

func main() {

    var n int

    fmt.Scan(&n)
    fmt.Println(faktorial(n))

}

func faktorial(n int) int {
    if n == 0 || n == 1 {
        return 1
    } else {
        return n * faktorial(n-1)
    }
}
```

Output:

```

1 package main
2
3 import "fmt"
4
5 func main() {
6
7     var n int
8
9     fmt.Scan(&n)
10    fmt.Println(faktorial(n))
11 }
12
13 func faktorial(n int) int {
14     if n == 0 || n == 1 {
15         return 1
16     } else {
17         return n * faktorial(n-1)
18     }
19 }

```

PROBLEMS 8 OUTPUT TERMINAL ...

rekursif> go run "c:\Users\vero0\go\pkg\mod\golang.org\x\text@v0.27.0\internal\language\modul 5 rekursif\guided 4.go"

12

479001600

PS C:\Users\vero0\go\pkg\mod\golang.org\x\text@v0.27.0\internal\language\modul 5 rekursif> █

Deskripsi Program:

- Program menghitung nilai faktorial bilangan menggunakan rekursif.
- Program menerima input sebuah bilangan n , kemudian memanggil fungsi faktorial(n).
- Fungsi akan mengalikan angka n dengan hasil faktorial dari angka sebelumnya ($n-1$) secara terus-menerus hingga mencapai kondisi berhenti, yaitu saat n bernilai 0 atau 1. Pada kondisi tersebut, fungsi mengembalikan nilai 1.

SOAL LATIHAN MODUL 5

1. Soal 1

Source Code:

```
package main

import "fmt"

func fibonacci(n int) int {
    if n == 0 {
        return 0
    } else if n == 1 {
        return 1
    } else {
        return fibonacci(n-1) + fibonacci(n-2)
    }
}

func main() {
    var jumlahSuku int

    fmt.Print("Masukkan jumlah suku: ")
    fmt.Scan(&jumlahSuku)

    fmt.Println("Deret Fibonacci:")

    for i := 0; i <= jumlahSuku; i++ {
        fmt.Print(fibonacci(i), " ")
    }
}
```

```

1:go soal 1.go >  fibonacci
5 func fibonacci(n int) int {
8     } else if n == 1 {
9         return 1
10    } else {
11        return fibonacci(n-1) + fibonacci(n-2)
12    }
13 }
14
15 func main() {
16     var jumlahSuku int
17
18     fmt.Print("Masukkan jumlah suku: ")
19     fmt.Scan(&jumlahSuku)
20
21     fmt.Println("Deret Fibonacci:")
22
23     for i := 0; i <= jumlahSuku; i++ {
24         fmt.Print(fibonacci(i), " ")
25     }

```

PROBLEMS

9

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

 Code











```

PS C:\Users\vero0\go\pkg\mod\golang.org\x\text@v0.27.0\internal\language\modul 5
rekursif> go run "c:\Users\vero0\go\pkg\mod\golang.org\x\text@v0.27.0\internal\la
nguage\modul 5 rekursif\soal 1.go"
Masukkan jumlah suku: 8
Deret Fibonacci:
0 1 1 2 3 5 8 13 21
PS C:\Users\vero0\go\pkg\mod\golang.org\x\text@v0.27.0\internal\language\modul 5
rekursif>

```

Program menampilkan deret Fibonacci sebanyak inputan, dengan menggunakan fungsi rekursif.

- Jika $n == 0$, hasilnya 0
- Jika $n == 1$, hasilnya 1

$$\text{fibonacci}(n) = \text{fibonacci}(n-1) + \text{fibonacci}(n-2)$$

Program menggunakan perulangan untuk mencetak hasil Fibonacci satu per satu.

2. Soal 2

Source Code:

```
package main

import "fmt"

func cetakBintang(baris int) {
    if baris == 0 {
        return
    }

    cetakBintang(baris - 1)

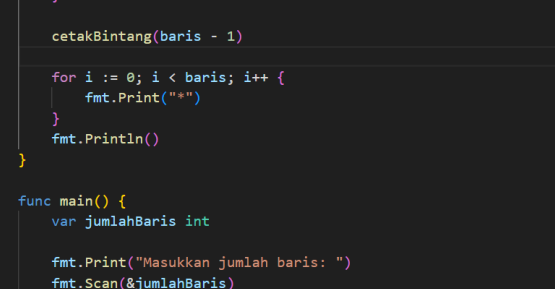
    for i := 0; i < baris; i++ {
        fmt.Print("*")
    }
    fmt.Println()
}

func main() {
    var jumlahBaris int

    fmt.Print("Masukkan jumlah baris: ")
    fmt.Scan(&jumlahBaris)

    cetakBintang(jumlahBaris)
}
```

Output:



The screenshot shows a Go IDE with a file named `soal 2.go`. The code defines a recursive function `cetakBintang` that prints a specified number of stars. The `main` function prompts the user to enter the number of stars, reads the input, and calls `cetakBintang` with the provided value.

```

1  soal 2.go > cetakBintang
2
3  func cetakBintang(baris int) {
4      //
5
6      //
7      //
8      //
9      //
10     cetakBintang(baris - 1)
11
12     for i := 0; i < baris; i++ {
13         fmt.Print("*")
14     }
15     fmt.Println()
16 }
17
18 func main() {
19     var jumlahBaris int
20
21     fmt.Print("Masukkan jumlah baris: ")
22     fmt.Scan(&jumlahBaris)
23
24     cetakBintang(jumlahBaris)
25 }

```

The terminal at the bottom shows the execution of the program:

```

nguage\modul 5 rekursif\soal 2.go"
Masukkan jumlah baris: 7
*
**
***
****
*****
*****
*****

```

Deskripsi Program:

- Program menampilkan pola bintang berbentuk segitiga. Program dimulai dengan menerima input dari user, yaitu 7 sebagai jumlah baris. Nilai ini kemudian dikirim ke fungsi cetakBintang(7).
- Fungsi bekerja secara rekursif dengan cara memanggil dirinya sendiri dengan nilai yang lebih kecil, yaitu dari 7 turun ke 6, 5, hingga mencapai 0. Saat nilai sudah 0, proses rekursif berhenti.
- Setelah itu, program mulai kembali. Pada saat kembali ini bintang dicetak. Pertama mencetak 1 bintang, lalu 2, kemudian 3, dan seterusnya hingga 7 bintang.

3. Soal 3

Source Code:

```
package main

import "fmt"

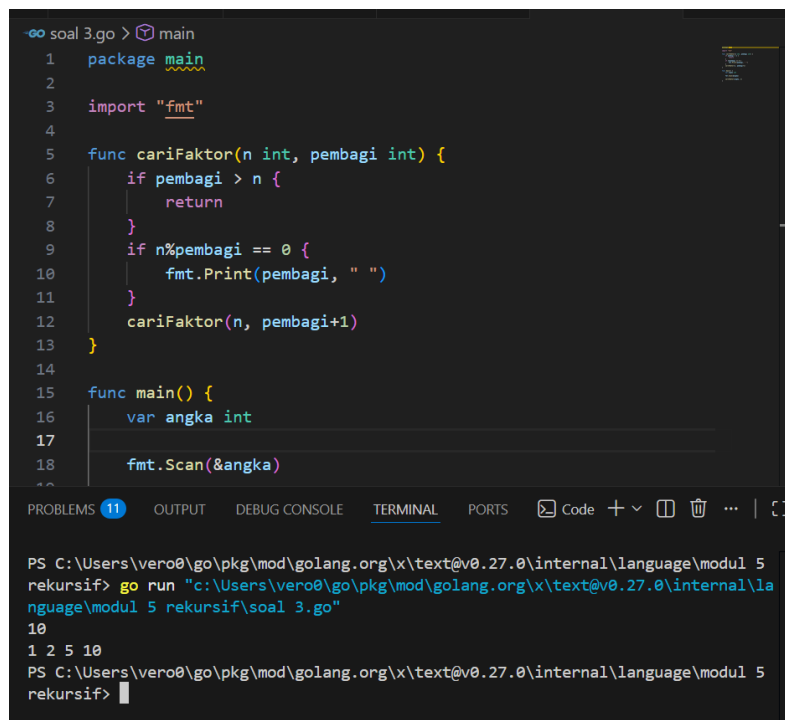
func cariFaktor(n int, pembagi int) {
    if pembagi > n {
        return
    }
    if n%pembagi == 0 {
        fmt.Print(pembagi, " ")
    }
    cariFaktor(n, pembagi+1)
}

func main() {
    var angka int

    fmt.Scan(&angka)

    cariFaktor(angka, 1)
}
```

Output:



```
soal 3.go > main
1 package main
2
3 import "fmt"
4
5 func cariFaktor(n int, pembagi int) {
6     if pembagi > n {
7         return
8     }
9     if n%pembagi == 0 {
10        fmt.Print(pembagi, " ")
11    }
12    cariFaktor(n, pembagi+1)
13 }
14
15 func main() {
16     var angka int
17
18     fmt.Scan(&angka)
```

PROBLEMS 11 OUTPUT DEBUG CONSOLE TERMINAL PORTS Code + -

```
PS C:\Users\vero0\go\pkg\mod\golang.org\x\text@v0.27.0\internal\language\modul 5
rekursif> go run "c:\Users\vero0\go\pkg\mod\golang.org\x\text@v0.27.0\internal\la
nguage\modul 5 rekursif\soal 3.go"
10
1 2 5 10
PS C:\Users\vero0\go\pkg\mod\golang.org\x\text@v0.27.0\internal\language\modul 5
rekursif>
```

Deskripsi Program:

- Program meminta inputan dari user, inputan 12.
- Nilai ini kemudian dikirim ke fungsi cariFaktor dengan pembagi awal 1.
- Fungsi akan mengecek satu per satu angka dari 1 sampai 12. Jika suatu angka bisa membagi 12 tanpa sisa, maka angka tersebut akan ditampilkan.
- Setiap kali selesai mengecek, fungsi akan memanggil dirinya sendiri dengan nilai pembagi yang bertambah satu, hingga pembagi lebih dari 12 dan proses berhenti.
- Hasilnya, program akan menampilkan semua faktor dari 12 secara berurutan

4. Soal 4

Source Code:

```
package main

import "fmt"

func barisan(n int) {
    if n == 1 {
        fmt.Print(1, " ")
        return
    }

    fmt.Print(n, " ")

    barisan(n - 1)
    fmt.Print(n, " ")
}

func main() {
    var angka int

    fmt.Scan(&angka)

    barisan(angka)
}
```

Output:

```

4:go soal 4.go > main
5:func barisan(n int) {
6:    fmt.Print(1, " ")
7:    return
8:}
9:
10:
11:    fmt.Print(n, " ")
12:
13:    barisan(n - 1)
14:    fmt.Print(n, " ")
15:}
16:
17:func main() {
18:    var angka int
19:
20:    fmt.Scan(&angka)
21:
22:    barisan(angka)
23:}

```

PROBLEMS 12 OUTPUT DEBUG CONSOLE TERMINAL PORTS Code + - [] [] [] [] [] []

```

PS C:\Users\vero0\go\pkg\mod\golang.org\x\text@v0.27.0\internal\language\modul 5
rekursif> go run "c:\Users\vero0\go\pkg\mod\golang.org\x\text@v0.27.0\internal\la
nguage\modul 5 rekursif\soal 4.go"
6
6 5 4 3 2 1 2 3 4 5 6
PS C:\Users\vero0\go\pkg\mod\golang.org\x\text@v0.27.0\internal\language\modul 5
rekursif>

```

Deskripsi Program:

- Program dimulai dengan menerima input dari user, misalnya **5**. Nilai ini dikirim ke fungsi barisan(6).
- Fungsi akan mencetak angka dari 6 ke bawah (6, 5, 4, 3, 2) sampai mencapai **1** sebagai kondisi berhenti.
- Saat sudah sampai 1, program mulai kembali ke atas (proses balik).
- Pada saat kembali inilah angka dicetak lagi, sehingga membentuk pola naik dari 2 sampai 6.

5. Soal 5

Source Code:

```
package main

import "fmt"

func ganjil(n int, i int) {
    if i > n {
        return
    }
    if i%2 != 0 {
        fmt.Print(i, " ")
    }

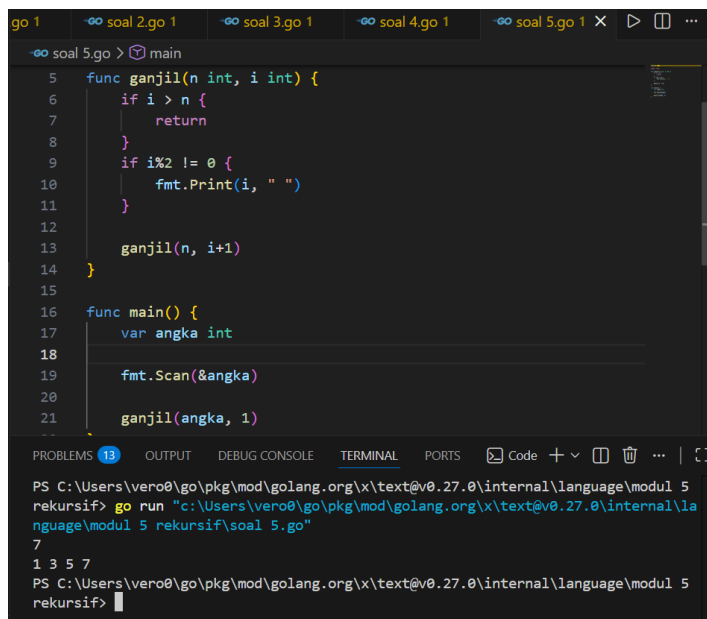
    ganjil(n, i+1)
}

func main() {
    var angka int

    fmt.Scan(&angka)

    ganjil(angka, 1)
}
```

Output:



```
go 1  -go soal 2.go 1  -go soal 3.go 1  -go soal 4.go 1  -go soal 5.go 1 X  ▶  □  ...  
-go soal 5.go > main  
5 func ganjil(n int, i int) {  
6     if i > n {  
7         return  
8     }  
9     if i%2 != 0 {  
10        fmt.Print(i, " ")  
11    }  
12  
13    ganjil(n, i+1)  
14 }  
15  
16 func main() {  
17     var angka int  
18  
19     fmt.Scan(&angka)  
20  
21     ganjil(angka, 1)  
22 }  
23  
PROBLEMS 13  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  Code + - □ □ □ ... | 🔍  
PS C:\Users\vero0\go\pkg\mod\golang.org\x\text@v0.27.0\internal\language\modul 5  
rekursif> go run "c:\Users\vero0\go\pkg\mod\golang.org\x\text@v0.27.0\internal\la  
nguage\modul 5 rekursif\soal 5.go"  
7  
1 3 5 7  
PS C:\Users\vero0\go\pkg\mod\golang.org\x\text@v0.27.0\internal\language\modul 5  
rekursif> |
```

Deskripsi Program:

- Program dimulai dengan menerima input dari user, inputan 7. Nilai ini dikirim ke fungsi ganjil dengan nilai awal 1.
- Fungsi akan mengecek setiap angka dari 1 sampai 7. Jika angka tersebut ganjil, maka akan ditampilkan.

6. Soal 6

Source Code:

```
package main

import "fmt"

func pangkat(x int, y int) int {
    if y == 0 {
        return 1
    }

    return x * pangkat(x, y-1)
}

func main() {
    var angka int
    var pangkatKe int

    fmt.Print("angka: ")
    fmt.Scan(&angka)

    fmt.Print("pangkat: ")
    fmt.Scan(&pangkatKe)

    fmt.Println("Hasil:", pangkat(angka, pangkatKe))
}
```

Output:

```

5 func pangkat(x int, y int) int {
10     return x * pangkat(x, y-1)
11 }
12
13 func main() {
14     var angka int
15     var pangkatKe int
16
17     fmt.Print("angka: ")
18     fmt.Scan(&angka)
19
20     fmt.Print("pangkat: ")
21     fmt.Scan(&pangkatKe)
22
23     fmt.Println("Hasil:", pangkat(angka, pangkatKe))
24 }
25

```

PROBLEMS 14 OUTPUT DEBUG CONSOLE TERMINAL PORTS     ... |

```

PS C:\Users\vero0\go\pkg\mod\golang.org\x\text@v0.27.0\internal\language\modul 5
rekursif> go run "c:\Users\vero0\go\pkg\mod\golang.org\x\text@v0.27.0\internal\la
nguage\modul 5 rekursif\soal 6.go"
angka: 5
pangkat: 3
Hasil: 125
PS C:\Users\vero0\go\pkg\mod\golang.org\x\text@v0.27.0\internal\language\modul 5
rekursif>

```

Deskripsi Program:

- Program dimulai dengan menerima dua input dari user, inputan **5** sebagai angka dan **3** sebagai pangkat.
- Nilai tersebut dikirim ke fungsi pangkat(5, 3). Fungsi akan mengalikan angka dengan dirinya sendiri secara berulang.
- Setiap pemanggilan, nilai pangkat akan dikurangi satu hingga mencapai **0** sebagai kondisi berhenti. Saat sudah 0, fungsi mengembalikan nilai 1, lalu hasil perkalian disusun saat proses kembali.
- Sehingga hasilnya $5 \times 5 \times 5 = 125$

DAFTAR PUSTAKA

Programiz. (n.d.). *Recursion in Programming*. <https://www.programiz.com/dsa/recursion>

Martins, M. D. (2012). *Distinctive Signatures of Recursion*. National Center for Biotechnology Information (NCBI). <https://pmc.ncbi.nlm.nih.gov/articles/PMC3367689/>